北京理工大學
BEIJING INSTITUTE OF TECHNOLOGY

2026

德以明理·学以精工

基于无人机的复杂场景中车牌识别方法研究

Dr. Yujie Fang (方宇杰)
TA：Junming Wu (吴浚铭)

BIT

北京理工大学

2024年11月

什么是图像?

- **定义为二维函数**f(x, y)**,其中，** x, y**是空间坐标，** f(x, y)**是点（**x, y**）的幅值**
- **灰度图像是一个二维灰度（或亮度）函数**f(x, y)
- **彩色图像由三个（如**RGB, HSV**）二维灰度（或亮度）函数**f(x, y)**组成**

图像的采样和量化

- 大多数传感器的输出是连续电压波形
- 为了产生一幅数字图像，需要把连续的感知数据转化为数字形式
- 这包括两种处理：取样和量化
- 取样：图像空间坐标的数字化
- 量化：图像函数值（灰度值）的数字化

什么是数字图像？

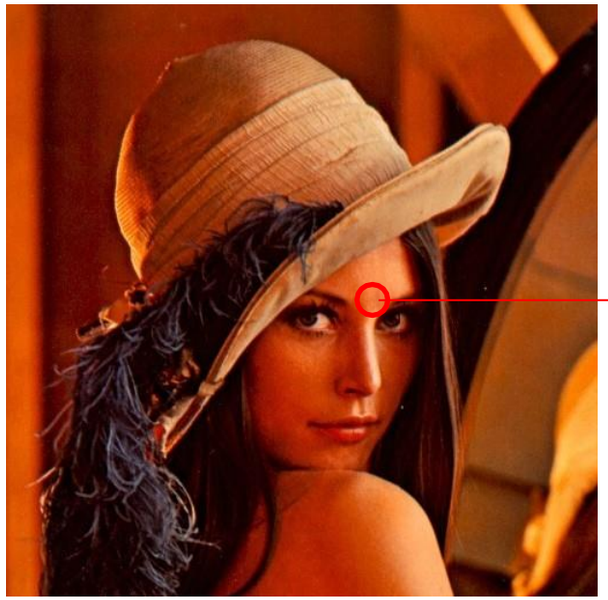
- 像素组成的二维排列，可以用矩阵表示

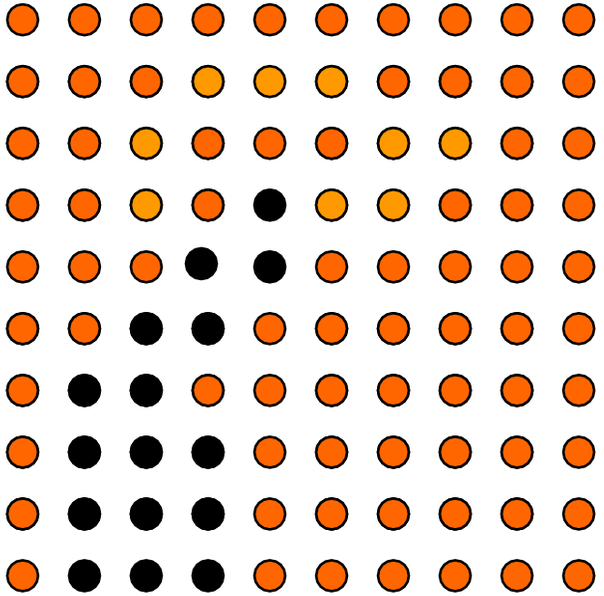
- 对于单色（灰度）图像而言，每个像素的亮度用一个数值来表示，通常数值范围在0到255之间，0表示黑、255表示白，其它值表示处于黑白之间的灰度

- 彩色图像可以用红R、绿G、蓝B三元组的二维矩阵来表示。 通常，三元组的每个数值也是在0到255之间，0表示相应的基色在该像素中没有，而255则代表相应的基色在该像素中取得最大值

什么是像素?

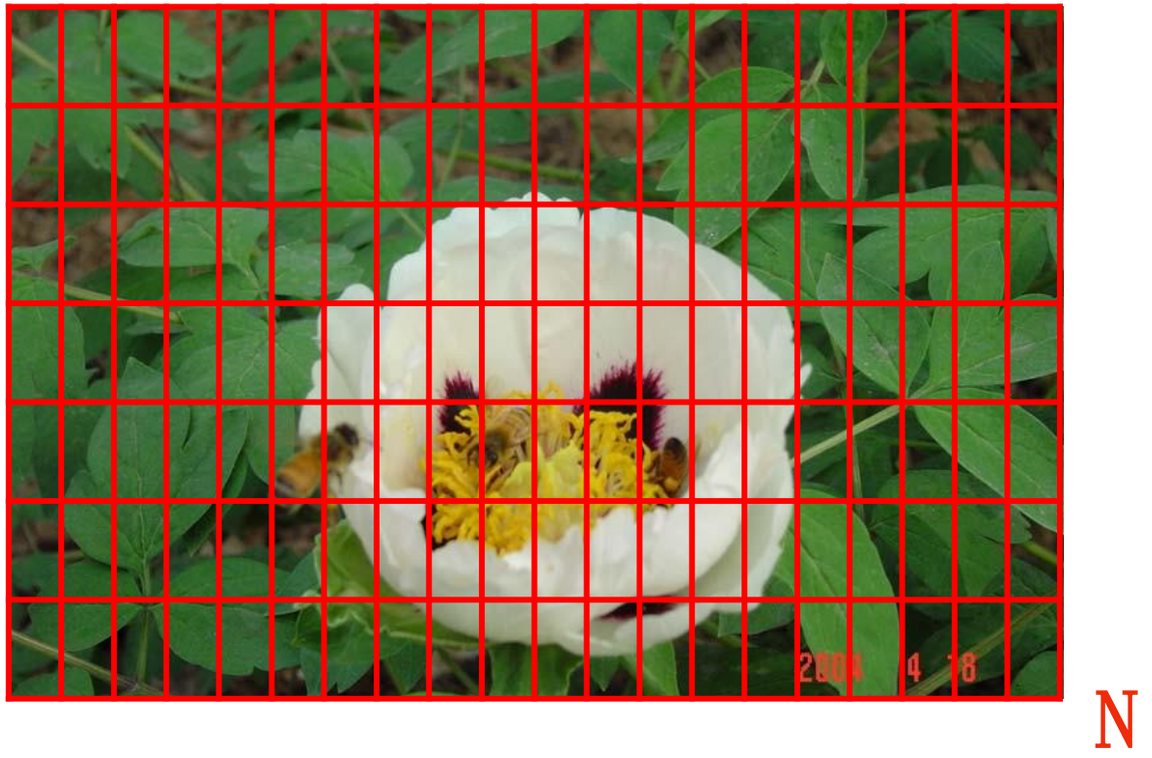
数字图像由二维的元素组成，每一个元素具有一个特定的位置（x, y）和幅值f(x, y)，这些元素就称为像素





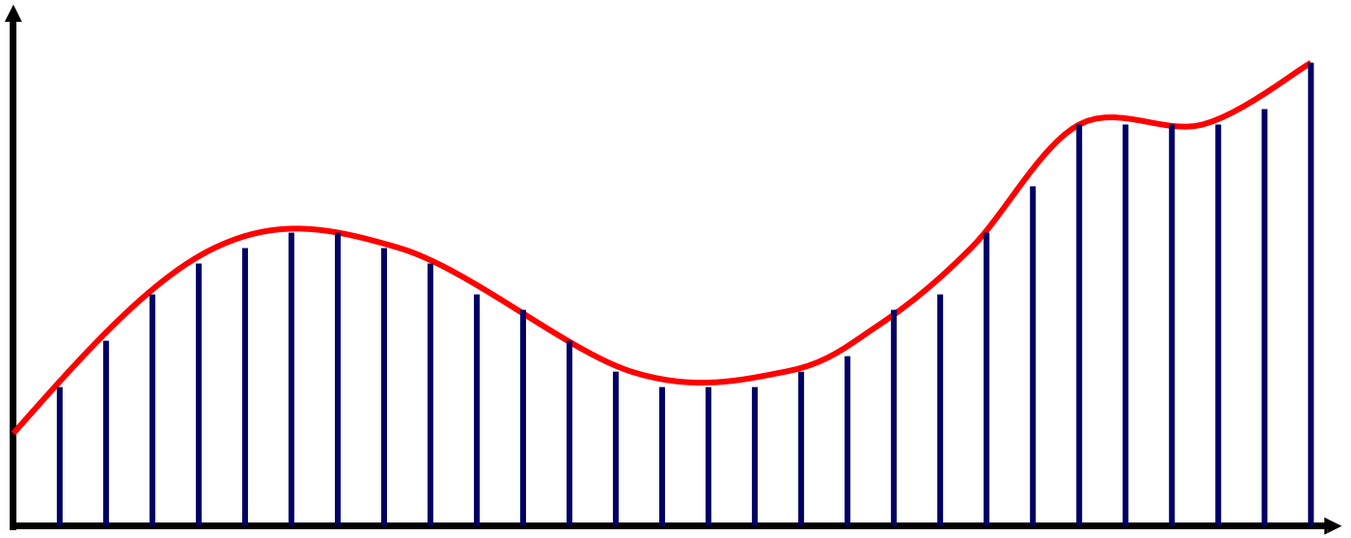
图像采样

- 空间坐标(x, y)的数字化被称为图像采样
- 确定水平和垂直方向上的像素个数N、 M



图像的量化

- 函数取值的数字化被称为图像的量化，如量化到256个灰度级



图像的采样与数字图像的质量

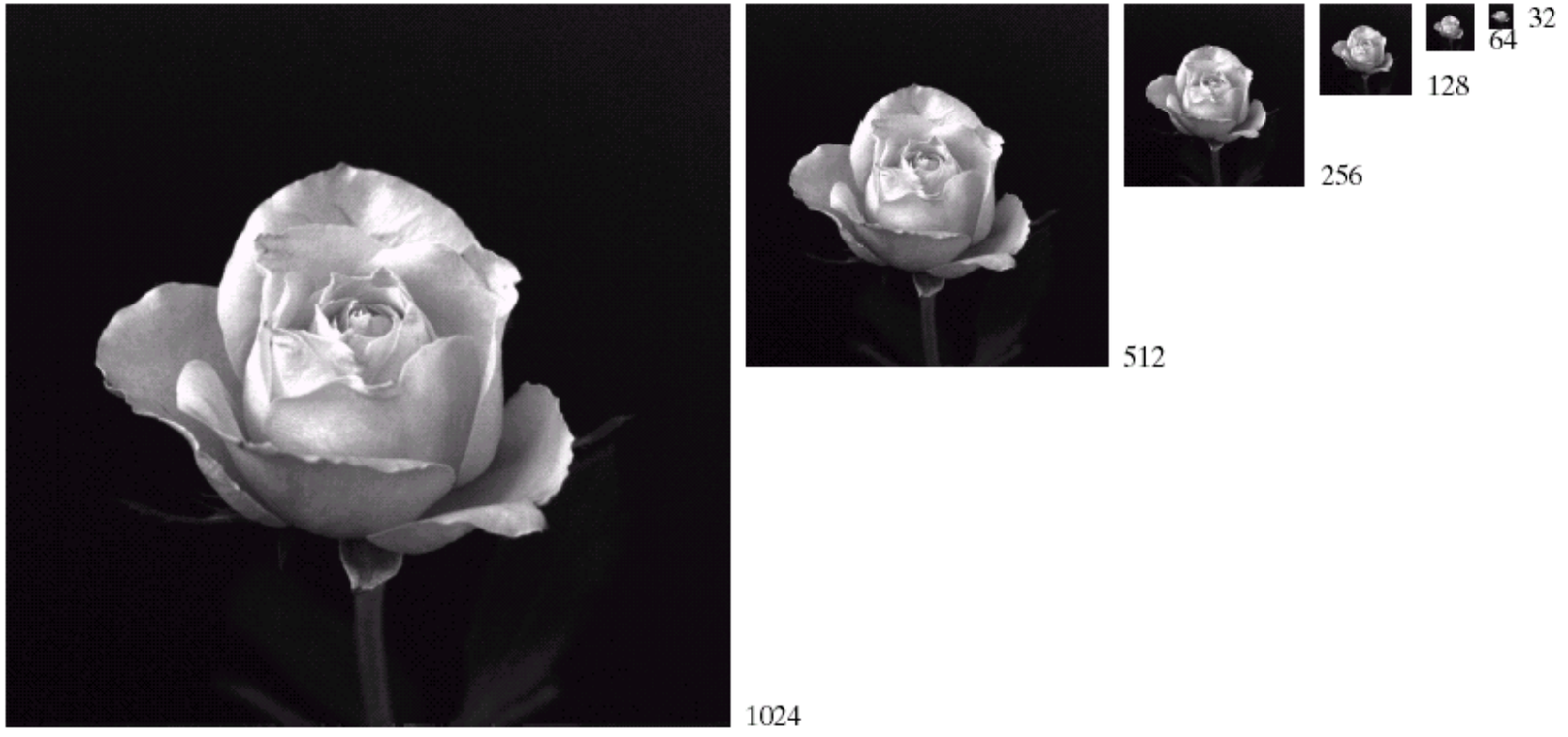


FIGURE 2.19 A 1024 × 1024, 8-bit image subsampled down to size 32 × 32 pixels. The number of allowable gray levels was kept at 256.

图像的采样与数字图像的质量

1024×1024　　512×512　　256×256

a b c
d e f

128×128　　64×64　　32×32

FIGURE 2.20 (a) 1024 × 1024, 8-bit image. (b) 512 × 512 image resampled into 1024 × 1024 pixels by row and column duplication. (c) through (f) 256 × 256, 128 × 128, 64 × 64, and 32 × 32 images resampled into 1024 × 1024 pixels.

图像灰度的概念

- 灰度级：表示像素明暗程度的整数量

  例如：像素的取值范围为0-255，就称该 图像为256个灰度级的图像

- 层次：表示图像实际拥有的灰度级的数量

  例如：具有32种不同取值的图像，可称 该图像具有32个层次

图像数据的实际层次越多，视觉效果就越好

256个层次的图像

64个层次的图像

16个层次的图像

数字图像的基本运算

从严格的意义上来说，各种图像处理方法都是一种图像运算方法。

从一般意义上来说，图像运算仅指对图像中的所有像素实施了相同处理的那些运算，比如对图像的点运算、直方图运算、代数运算、几何运算、灰度插值运算等。

灰度反转

灰度反转

黑白图像的反转就是使灰度值为1的像素值变成0，使灰度值为0的像素值变成1。

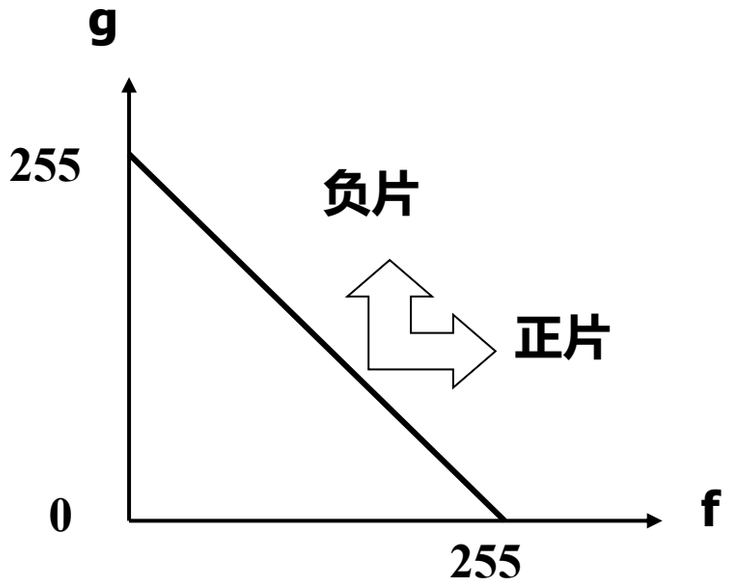
对于256灰度级图像来说，图像的灰度反转值就是用255分别减去原图像的各个像素的灰度值。

设图像的灰度级为L，则图像的灰度反转可表示为：

$$g(x,y) = \mathrm{L} - 1 - \mathrm{f}(\mathrm{x},\mathrm{y})$$

灰度反转

256灰度级图像的灰度反转如图1所示。



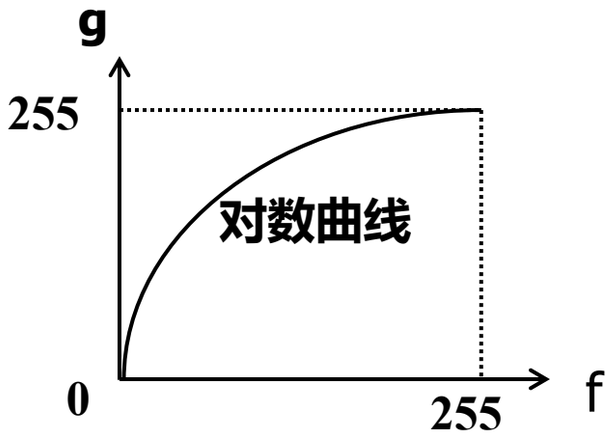
对数变换

对数变换

对原图像f(x, y)进行对数变换的解析式可表示为：

$$g(x,y) = \mathrm{c} \cdot \log(1 + \mathrm{f}(\mathrm{x},\mathrm{y}))$$

对数变换曲线如下图所示：



对数变换

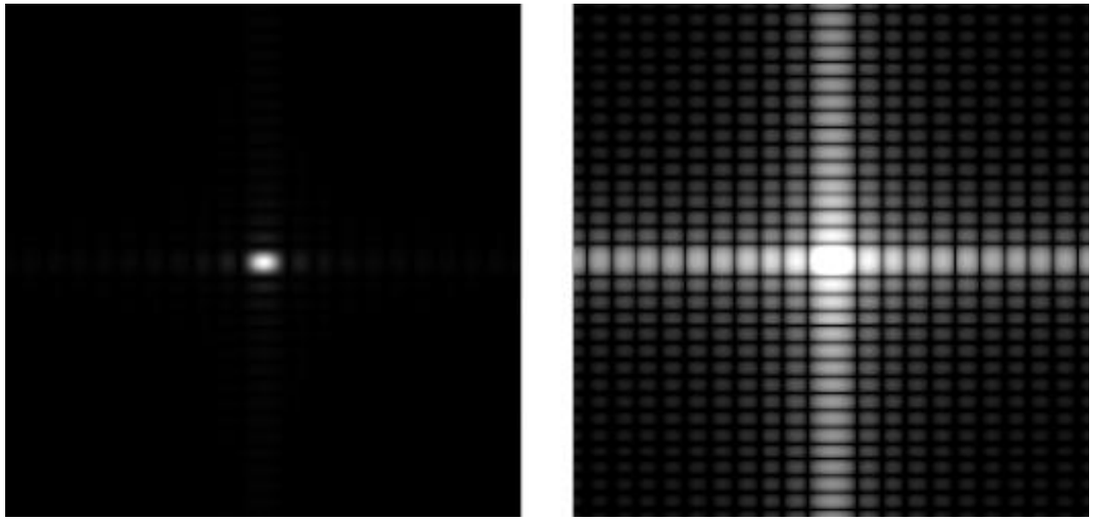
对数变换的作用：

（1）通过对图像的灰度值的动态范围进行压缩，来主要用于调高输入图像的低灰度值。

（2）人的视觉感觉与进入人眼的光的强度成对数关系，常先给图像进行对数变换后再显示输出。

对数变换

傅里叶频谱的显示一般都就是通过这种方式的调整来进行显示的。



(a) 图像的傅里叶频谱　　(b) 对(a)进行c=1的对数变换的结果

灰度直方图

灰度直方图

灰度图像的直方图是一种表示数字图像中各级灰度值及其出现频数的关系的函数，一般用一个二维坐标来表示。

描述灰度图像直方图的二维坐标的横坐标用于表示像素的灰度级别，纵坐标用于表示该灰度出现的频数（像素的个数）。

灰度直方图

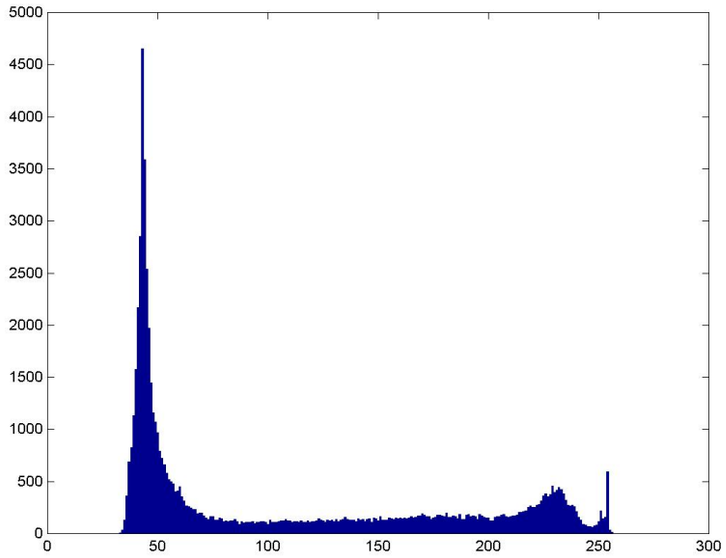
设一幅数字图像的灰度级范围为[0，L−1]，则该图像的灰度直方图可定义为：

$$h(r_k) = n_k \qquad r_k = 0,\ 1,\ 2,\ \ldots,\ L\text{-}1 \qquad (3.3)$$

其中，r_k表示第 k 级灰度值；n_k表示图像中灰度值为 r_k的像素的个数；$h(r_k)$是灰度图像的直方图函数。

灰度直方图

- 灰度直方图表明图像中每一个灰度级有多少个象素



由 Matlab直方图计算程序 计算所得

灰度直方图

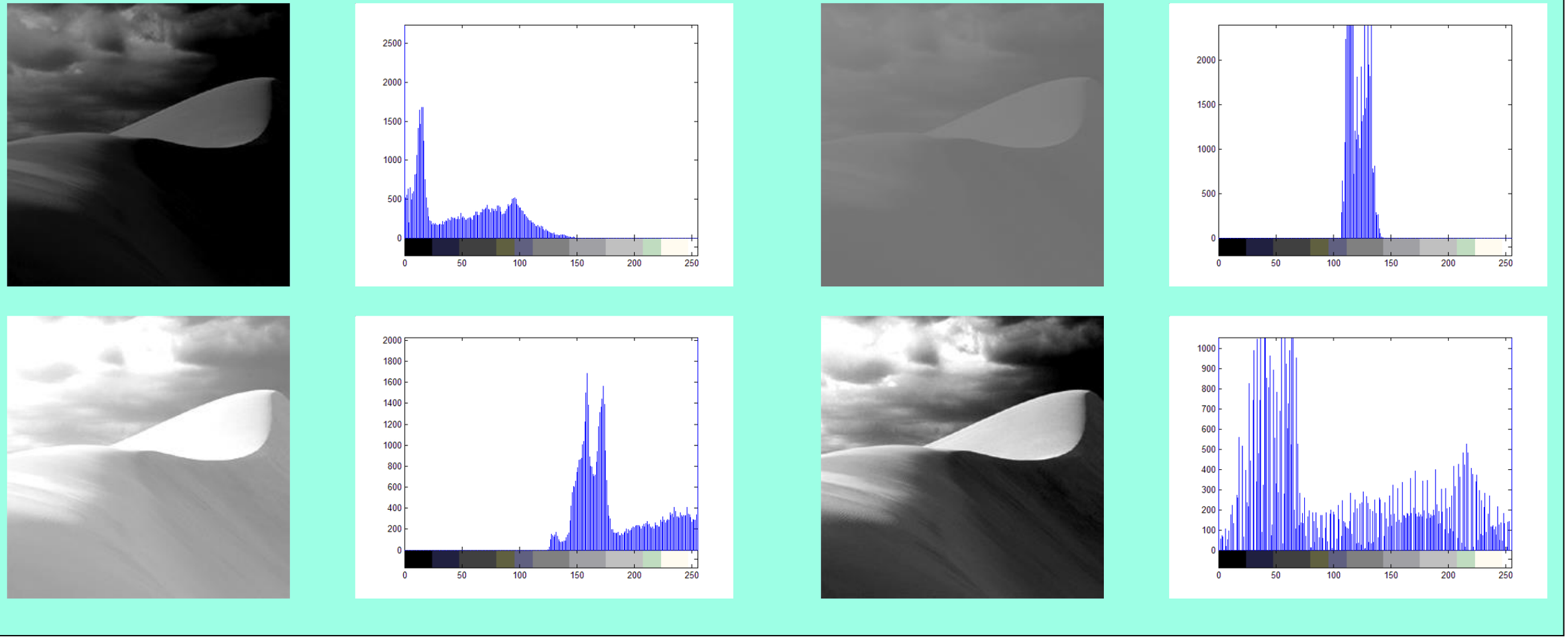


图3.3 四种基本图像类型图像及其灰度直方图

德以明理 学以精工

归一化灰度图像直方图的引入：

由于式（3.3）所定义的灰度直方图反映的是图像中各灰度的实际出现频数，这样当某个灰度值的频数（计数值）远远大于其它灰度值的频数时，根据图像的某个或某些像素出现的最大频数来确定直方图的纵坐标的最大尺度既不方便也不太现实，所以就引入了归一化直方图的概念

$$h(r_k) = n_k \qquad r_k=0,\ 1,\ 2,\ \ldots,\ \text{L-1} \qquad (3.3)$$

灰度直方图

设 r_k 为图像 $f(x,y)$ 的第 k 级灰度值，n_k 是图像 $f(x,y)$ 中具有灰度值 r_k 的像素的个数，n 是图像 $f(x,y)$ 中的像素总数，则图像 $f(x,y)$ 的灰度直方图定义为：

$$P(r_k)=\frac{n_k}{n} \qquad 0 \le r_k \le 1;\ k=0,\ 1,\ \ldots,\ \text{L-1}$$

灰度直方图

灰度直方图具有如下一些特征：

（1）直方图仅能描述图像中每个灰度值具有的像素个数，不能表示图像中每个像素的位置(空间)信息 ；

（2）任一特定的图像都有惟一的直方图，不同的图像可以具有相同的直方图；

（3）对于空间分辨率为M×N，且灰度级范围为[0, L-1]的图像，有关系：

$$\sum_{j=0}^{L-1} h(j) = \mathrm{M}\times\mathrm{N}$$

（4）如果一幅图像由两个不连接的区域组成，则整幅图像的直方图等于两个不连接的区域的直方图之和。

图像代数运算

加运算

图像相加是通过对两幅大小相同的图像对应位置像素的相加运算，以产生一幅新的含有两幅图像信息的图像的方法。图像相加也称为图像合成。

加运算

设 $f_1(x,y)$ 和 $f_2(x,y)$分别表示大小为 $M \times N$ 的两幅输入图像，图像 $f_1(x,y)$ 和 $f_2(x,y)$ 图像相加后得到的结果输出图像为 $g(x,y)$，且：

$x \in$[0，M-1]，$y \in$[0，N-1]，则两幅图像的相加运算可表示为：

$$g(x,y) = f_1(x,y) + f_2(x,y)$$

德以明理 学以精工

1、两幅图像内容的叠加/合成

两幅256灰度级图像对应坐标位置像素值的相加，其结果必然会超过灰度的最大表示范围256。

常用的处理方法是将两幅图像的灰度值折半相加，如式（3.7）所示；其实质就是求两幅图像像素灰度值相加后的平均值作为相加结果。

$$g(x,y)=IntergerRound(\frac{1}{2}f_1(x,y)+\frac{1}{2}f_2(x,y))$$

加运算

+

=

加运算

+

=

1、两幅图像内容的叠加/合成

式(3.7)可以推广到两幅灰度图像按不同比例灰度值的叠加/合成，如式（3.8）：

$$g(x,y)=\alpha f_1(x,y)+\beta f_2(x,y)$$

其中：

$$\alpha+\beta=1$$

2、多幅图像的叠加

$$g(x,y)=\frac{1}{N}f_i(x,y);\quad i=1,2,\cdots,N$$

多幅灰度图像的叠加实质上是一种灰度图像的噪声消除方法。

加运算

设 $f_1(x,y)$ 和 $f_2(x,y)$分别表示大小为 $M \times N$ 的两幅输入图像，从图像 $f_1(x,y)$ 中的各位置的像素中减去的相应位置的像素值后，得到的结果输出图像为 $g(x,y)$，且 $x \in [0, M-1]$，$y \in [0, N-1]$，则两幅图像的相减运算可表示为：

$$g(x,y) = f_1(x,y) - f_2(x,y)$$

减运算

—

=

减运算

当两幅256灰度级图像对应坐标位置像素值相减的结果大于或等于零时，则取其为结果图像中的像素的灰度值；

当相减结果小于零时，一般都是取零为结果值。

对于某些特殊的应用目的，也可以取其绝对值为结果值。

图像相减运算的典型应用是图像的变化检测。

图像几何运算

图像的几何运算又称为图像的几何变换，包括：

1、图像的平移变换；

2、图像的旋转变换；

3、图像镜像；

4、图像转置；

5、图像的缩放。

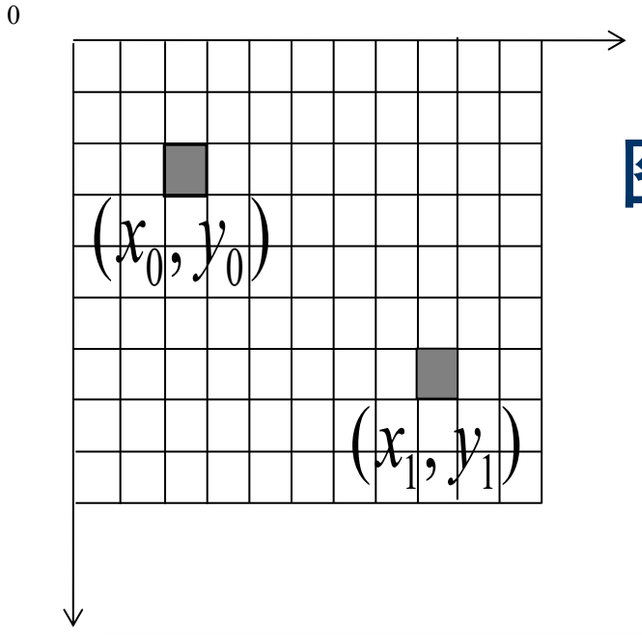
图像平移变换

图像平移变换是指将一幅图像或一幅图像中的子图像块（以下简称为图像块）中的所有像素点，都按指定的X 方向偏移量△X 和Y 方向偏移量△Y 进行移动。

图像平移变换

设初始坐标为 (x_0, y_0) 的像素平移 $(\Delta x, \Delta y)$ 后的坐标为 (x_1, y_1) ，则有

$$\begin{cases} x_1 = x_0 + \Delta x \\ y_1 = y_0 + \Delta y \end{cases} \tag{3.11}$$



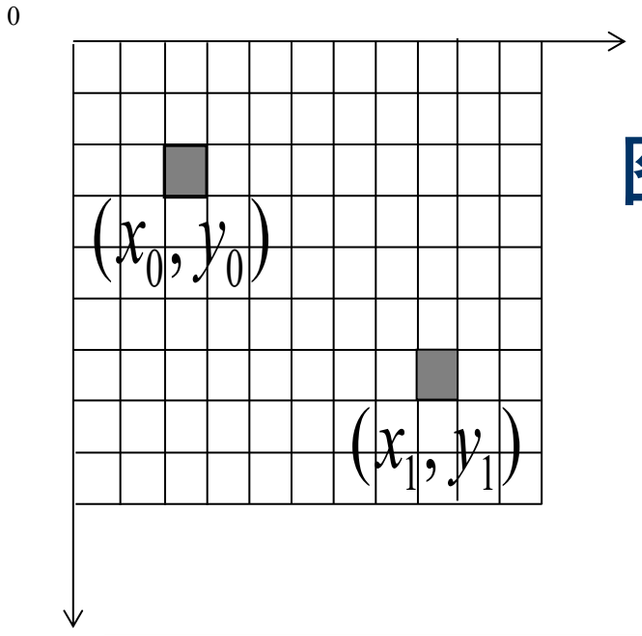
图像平移变换式的矩阵形式为

$$\begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & \Delta x \\ 0 & 1 & \Delta y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \\ 1 \end{bmatrix} \tag{3.12}$$

图像平移变换

设初始坐标为 (x_0, y_0) 的像素平移 $(\Delta x, \Delta y)$ 后的坐标为 (x_1, y_1) ，则有

$$\begin{cases} x_1 = x_0 + \Delta x \\ y_1 = y_0 + \Delta y \end{cases} \tag{3.11}$$



图像平移变换的逆变换矩阵形式为

$$\begin{bmatrix} x_0 \\ y_0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & -\Delta x \\ 0 & 1 & -\Delta y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} \tag{3.13}$$

图像平移变换

德以明理　学以精工

实现方式：

（1）图像块平移

原图像　　平移图像子块后的图像

图像平移变换

实现方式：

（2）整幅图像平移：图像平移保持原幅面大小；被移出的部分被截掉。

移动前的图像

平移后的图像

图像旋转变换

图像旋转变换是指，以图像的中心为原点，将图像中的所有像素（也即整幅图像）旋转一个相同的角度。

图像旋转变换

实现方式：

（1）旋转后的图像幅面被放大；

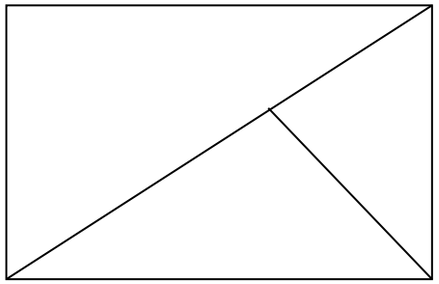
（2）保持图像旋转前后的幅面大小，把旋转后图像被转出原幅面大小的那部分截断。

图像旋转变换

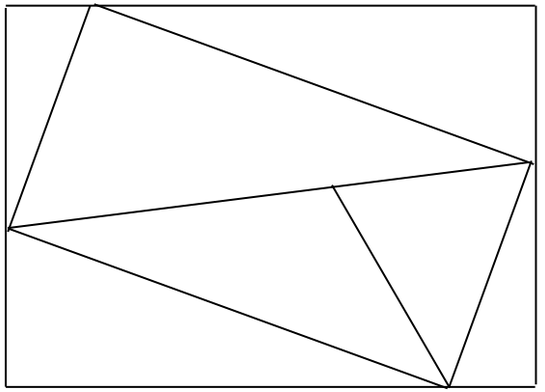
实现方式：

（1）旋转后的图像幅面被放大 √

（2）保持图像旋转前后的幅面大小，把旋转后图像被转出原幅面大小的那部分截断。



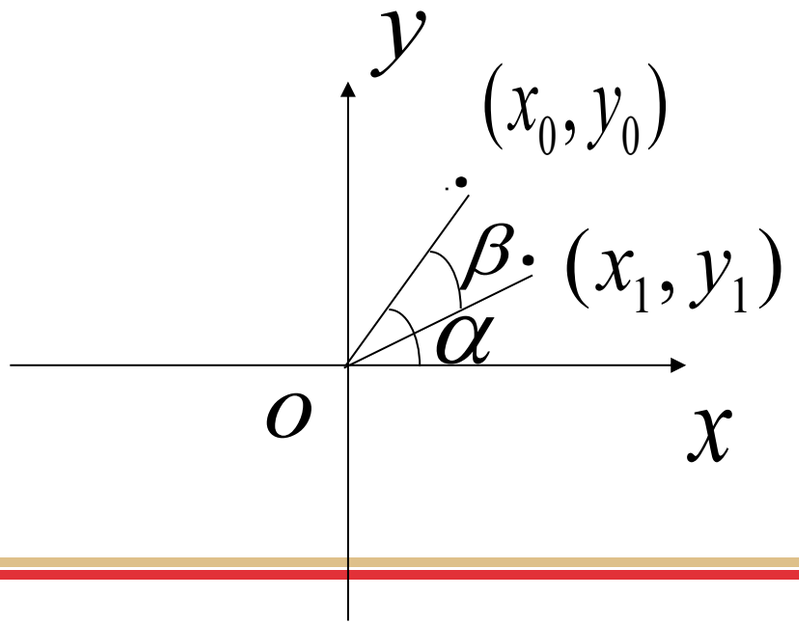
旋转前的图像



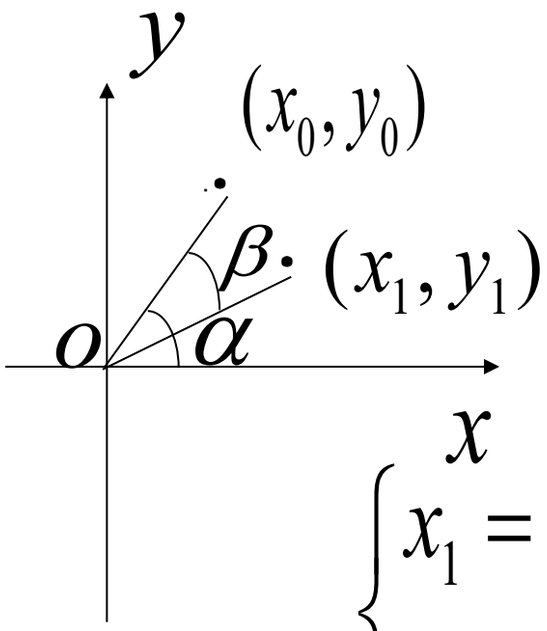
旋转后的图像

1. 基于xoy平面坐标系的点旋转变换

设位于 (x_0, y_0) 处的点到坐标原点的直线 r 与轴 x 的夹角为 α，直线 r 顺时针旋转 β 角度后使位于 (x_0, y_0) 处的点被旋转至 (x_1, y_1) 处 。



图像旋转变换



旋转前：

$$\begin{cases} x_0 = r\cos\alpha \\ y_0 = r\sin\alpha \end{cases}$$

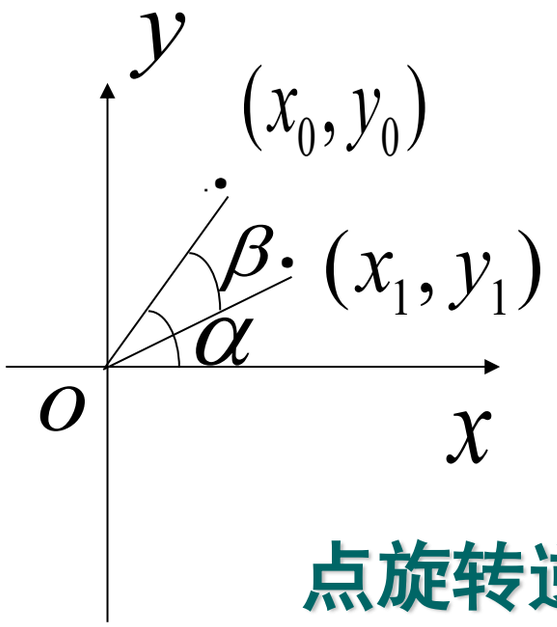
旋转后：

$$\begin{cases} x_1 = r\cos(\alpha-\beta) = r\cos\alpha\cos\beta + r\sin\alpha\sin\beta \\ y_1 = r\sin(\alpha-\beta) = r\sin\alpha\cos\beta - r\cos\alpha\sin\beta \end{cases}$$

旋转前式代入旋转后式（点旋转变换表示形式）：

$$\begin{cases} x_1 = x_0\cos\beta + y_0\sin\beta \\ y_1 = -x_0\sin\beta + y_0\cos\beta \end{cases}$$

图像旋转变换



点旋转变换的矩阵表示形式：

$$\begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\beta & \sin\beta & 0 \\ -\sin\beta & \cos\beta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \\ 1 \end{bmatrix} \tag{3.17}$$

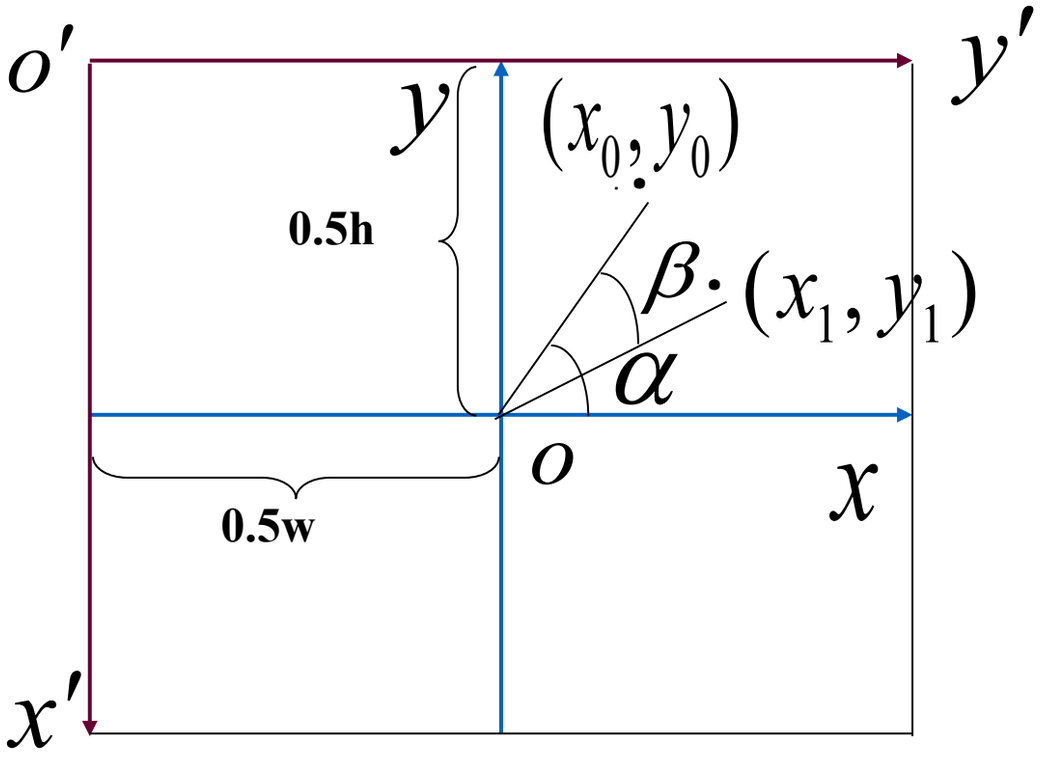
点旋转逆变换的矩阵表示形式：

$$\begin{bmatrix} x_0 \\ y_0 \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\beta & -\sin\beta & 0 \\ \sin\beta & \cos\beta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} \tag{3.18}$$

2. 基于（图像显示坐标的）图像像素点的旋转变换

由于图像中像素点的旋转变换是基于图像的显示坐标的，所以还需将得到的点旋转变换的矩阵表示形式（变换结果）映射到图像的显示坐标中。

2. 基于(图像显示坐标的)图像像素点的旋转变换

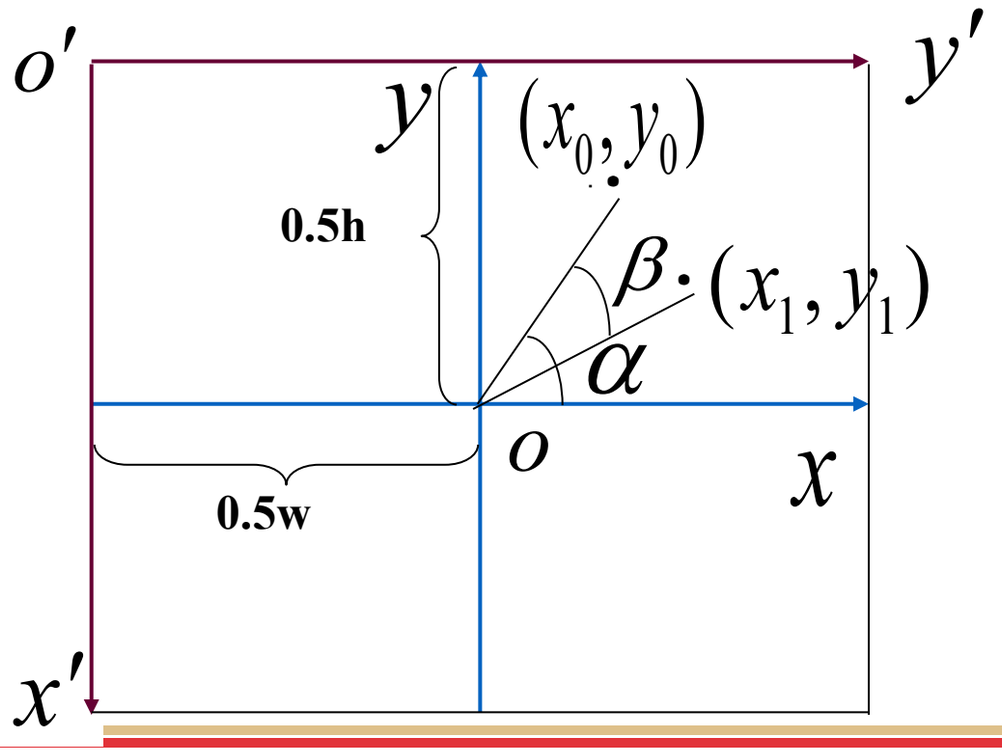


图像的像素点与图像的显示坐标和 xoy 平面坐标的关系示例

假设图像的宽度width用w 表示，图像的高度high用表示h ，由图可知：在原点 o 位于左上角 x' 轴方向朝下 y' 轴方向朝右的图像显示坐标中，如果用 (x_0', y_0') 表示 (x_0, y_0) 在图像显示坐标 $x'o'y'$ 中的位置，用 (x_1', y_1') 表示 (x_1, y_1) 在图像显示坐标 $x'o'y'$ 中的位置，则有：

$$\begin{cases} x_0' = 0.5h - y_0 \\ y_0' = 0.5w + x_0 \end{cases}$$

$$\begin{cases} x_1' = 0.5h - y_1 \\ y_1' = 0.5w + x_1 \end{cases}$$



上两式的矩阵形式分别为：

$$\begin{bmatrix} x_0' \\ y_0' \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0.5h \\ 1 & 0 & 0.5w \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \\ 1 \end{bmatrix} \tag{3.21}$$

$$\begin{bmatrix} x_1' \\ y_1' \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0.5h \\ 1 & 0 & 0.5w \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} \tag{3.22}$$

图像旋转变换

将式（3.17）：

（基于**xoy**坐标的点旋转变换矩阵表示形式）

$$\begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\beta & \sin\beta & 0 \\ -\sin\beta & \cos\beta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \\ 1 \end{bmatrix}$$

代入式(3.22)，可得图像像素点的旋转变换公式为：

$$\begin{bmatrix} x_1' \\ y_1' \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0.5h \\ 1 & 0 & 0.5w \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\beta & \sin\beta & 0 \\ -\sin\beta & \cos\beta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \\ 1 \end{bmatrix}$$

图像旋转变换

将式（3.18）：

（基于**xoy**坐标的点旋转逆变换矩阵表示形式）

$$\begin{bmatrix} x_0 \\ y_0 \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\beta & -\sin\beta & 0 \\ \sin\beta & \cos\beta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix}$$

代入式(3.21)，可得图像像素点的旋转逆变换公式为：

$$\begin{bmatrix} x_0' \\ y_0' \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0.5h \\ 1 & 0 & 0.5w \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\beta & -\sin\beta & 0 \\ \sin\beta & \cos\beta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix}$$

图像镜像变换

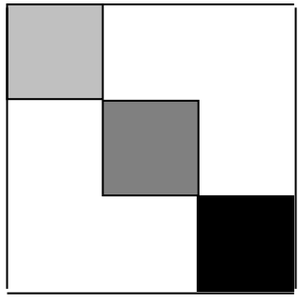
图像镜像变换分为：

（1）图像水平镜像变换

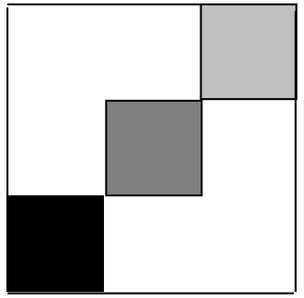
（2）图像垂直镜像变换

1. 图像水平镜像

◆ 图像水平镜像是指以原图像为参照，使原图像和水平镜像结果图像与虚拟的垂直轴成对称关系。



原图像



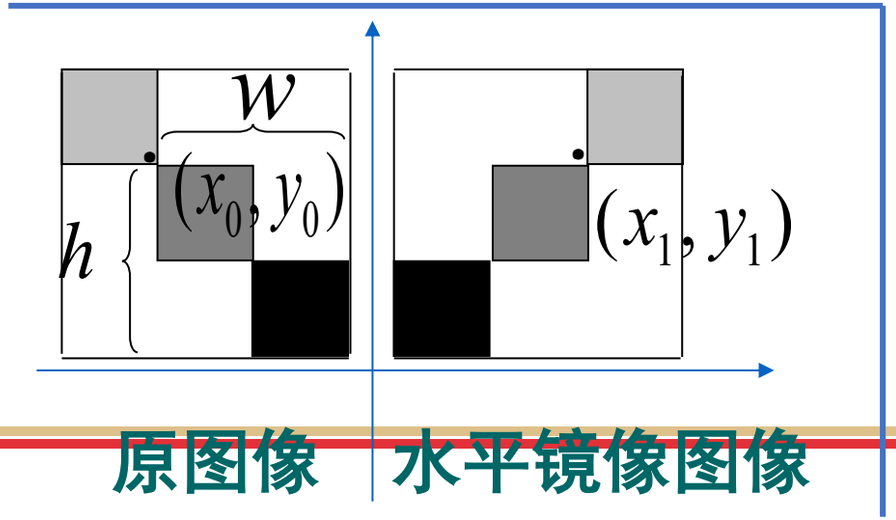
水平镜像结果图像

图像镜像变换

1. 图像水平镜像

设图像的高度high为 h ，宽度width为 w ；原图像中位于 (x_0, y_0) 处的像素点，经水平镜像后在水平镜像结果图像上的对应像素点为 (x_1, y_1) 。则**图像水平镜像变换**可表示为：

$$\begin{cases} x_1 = w - x_0 \\ y_1 = y_0 \end{cases}$$



图像镜像变换

1. 图像水平镜像

图像水平镜像变换：

$$\begin{cases} x_1 = w - x_0 \\ y_1 = y_0 \end{cases}$$

图像水平镜像变换矩阵表示形式为：

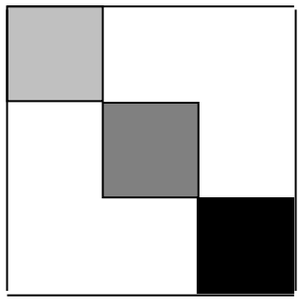
$$\begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & w \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \\ 1 \end{bmatrix}$$

图像水平镜像变换的逆变换的矩阵表示形式为：

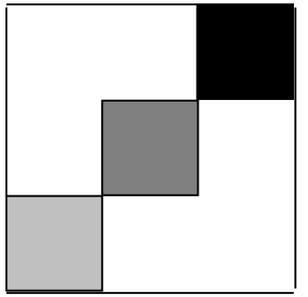
$$\begin{bmatrix} x_0 \\ y_0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & w \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix}$$

2. 图像垂直镜像

◆ 图像垂直镜像是指以原图像为参照，使原图像和垂直镜像结果图像与虚拟的水平轴成对称关系。



原图像

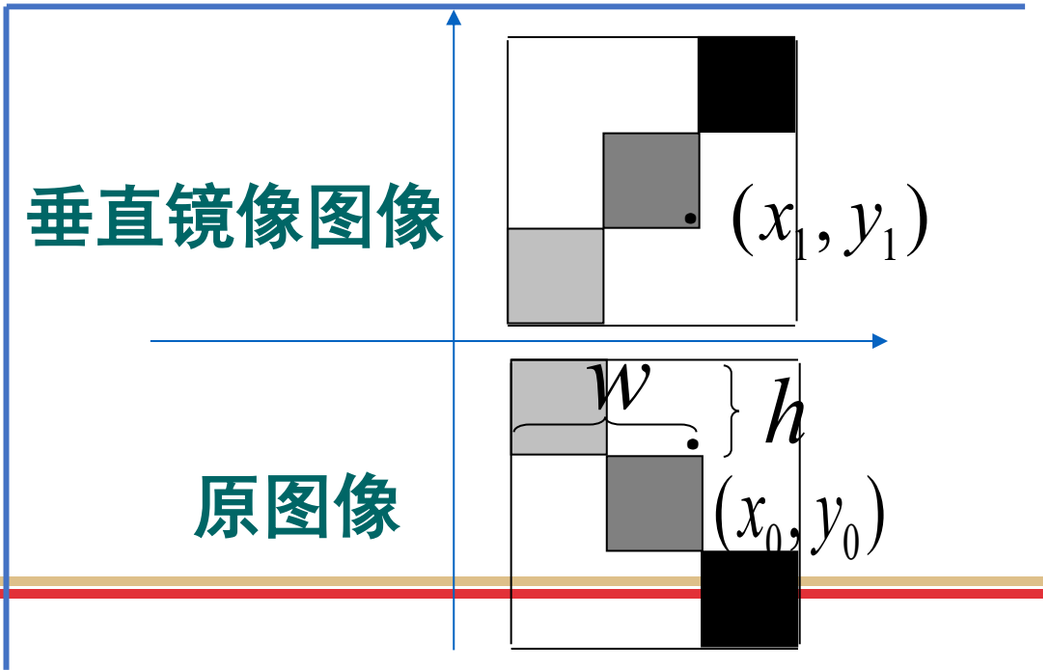


垂直镜像结果图像

2. 图像垂直镜像

设图像的高度high为 h ，宽度width为 w ；原图像中位于 (x_0, y_0) 处的像素点，经垂直镜像后在垂直镜像结果图像上的对应像素点为 (x_1, y_1) 。则**图像垂直镜像变换**可表示为：

$$\begin{cases} x_1 = x_0 \\ y_1 = h - y_0 \end{cases}$$



德以明理 学以精工

2. 图像垂直镜像

图像垂直镜像变换：

$$\begin{cases} x_1 = x_0 \\ y_1 = h - y_0 \end{cases}$$

图像垂直镜像变换矩阵表示形式为：

$$\begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & h \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \\ 1 \end{bmatrix}$$

图像垂直镜像变换的逆变换的矩阵表示形式为：

$$\begin{bmatrix} x_0 \\ y_0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & h \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix}$$

图像镜像变换

图像转置变换是指将图像显示坐标的x 轴与y轴对换。

设原图像位于 (x_0, y_0) 处的像素点，经图像转置变换后在转置变换结果图像上的对应像素点为 (x_1, y_1)。则图像转置变换可表示为：

$$\begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \\ 1 \end{bmatrix}$$

图像转置变换

图像转置示例：

转置前图像

转置后的图像

特别注意：

图像的转置变换与图像的旋转变换是不一样的。

原图像不论是顺时针旋转90°，还是逆时针旋转90°都不会得到图像转置后的结果。

图像转置变换

图像转置变换可表示为：

$$\begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \\ 1 \end{bmatrix}$$

同理，有图像转置变换的逆变换：

$$\begin{bmatrix} x_0 \\ y_0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix}$$

图像缩放

图像缩放是指对图像进行缩小或放大，也即对数字图像的大小进行调整的过程。

图像缩放

设原图像中位于 (x_0, y_0) 处的像素点，经对原图像的行和列按相同比例 r 缩放后，在缩放后的结果图像上的对应像素点为 (x_1, y_1) 。则图像缩放前后像素点的坐标可表示为：

$$\begin{cases} x_1 = rx_0 \\ y_1 = ry_0 \end{cases}$$

图像缩放的矩阵表示形式为：

$$\begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = \begin{bmatrix} r & 0 & 0 \\ 0 & r & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \\ 1 \end{bmatrix}$$

德以明理 学以精工

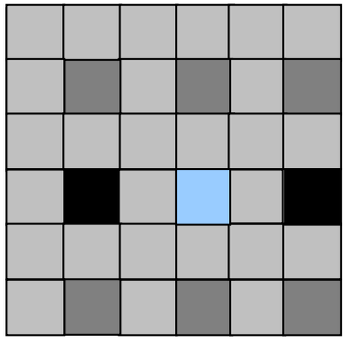
1. 图像缩小

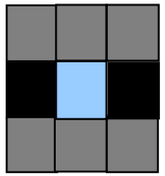
图像缩小的目的：**一是**为了使缩小后的图像符合显示区域的大小要求；**二是**为了生成原图像的缩略图。

最简单的图像缩小方法是：将图像的行和列都缩小一半，从整体上看就是将原图像缩小到原来大小的四分之一。

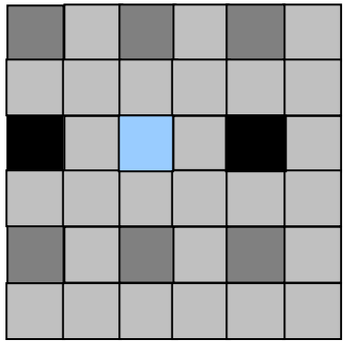
一般小方法是：按整数比例缩小图像。

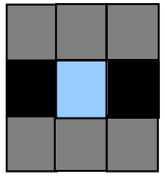
图像缩放





仅取偶数行和偶数列像素缩小原图像





仅取奇数行和奇数列像素缩小原图像

2. 图像放大

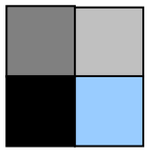
放大图像的目的一般是为了使放大后的图像更好地显示在更高分辨率的显示设备上。

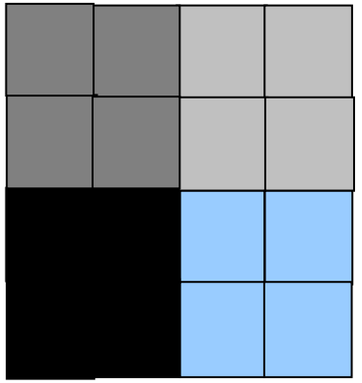
放大图像的方法较多，也较复杂。典型的图像放大方法是最近邻域插值法。

2. 图像放大

1）按整数倍放大图像的最近邻域插值法

基本思想：将原图像中的每一个原像素原封不动地复制映射到放大后的新图像中的四个像素中。





2) 按非整数倍放大图像的最近邻域插值法

设：放大前的原图像在显示坐标的X方向和Y方向的坐标值分别用X_{old}和Y_{old}表示；图像高度（X方向）和宽度（Y方向）分别用H_{old}和W_{old}表示。

放大后的目标图像在显示坐标的X方向和Y方向的坐标值分别用X_{new}和Y_{new}表示；图像高度（X方向）和宽度（Y方向）分别用H_{new}和W_{new}表示。

则最近邻域插值法的原图像和目标图像的坐标关系为

$$\begin{cases} x_{old} = x_{new} * (h_{old} / h_{new}) \\ y_{old} = y_{new} * (w_{old} / w_{new}) \end{cases}$$

2) 按非整数倍放大图像的最近邻域插值法

例3.1 设已知有一个3×3的灰度图像如下左图。按最近邻域插值法（非整数倍放大图像）放大为 4X4的图像如下右图。

234	38	22
67	44	12
89	65	63

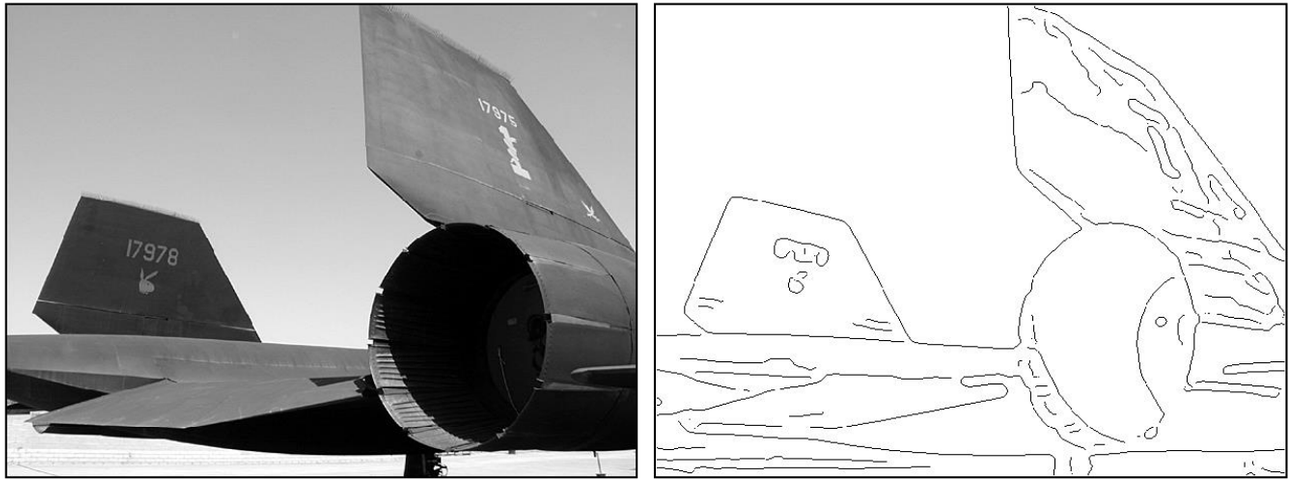
原图像

234	38	22	22
67	44	12	12
89	65	63	63
89	65	63	63

放大后的图像

用定向滤波器检测边缘

- 边缘在人类的视觉系统中扮演着非常重要的角色



(a) (b)

- 边缘可以粗略地描述为图像中那些沿某一特定方向局部强度变化显著的位置
- 在数学上，这个与空间距离相关的变化量，就是函数的一阶导数

图像滤波器

- 图像函数$I(u,v)$沿u轴和v轴的偏导数为

$$\frac{\partial I}{\partial u}(u,v) \qquad \text{and} \qquad \frac{\partial I}{\partial v}(u,v)$$

- 函数I在点(u,v)处的梯度向量（或简称为梯度）为：

$$\nabla I(u,v) = \begin{bmatrix} \frac{\partial I}{\partial u}(u,v) \\ \frac{\partial I}{\partial v}(u,v) \end{bmatrix}$$

- 其模值 $$|\nabla I|(u,v) = \sqrt{\left(\frac{\partial I}{\partial u}(u,v)\right)^2 + \left(\frac{\partial I}{\partial v}(u,v)\right)^2}$$

不会随图像旋转而变化，对各向同性的边缘定位起着重要作用，因此是许多实用边缘检测方法的基础

图像滤波器

- Prewitt算子与Sobel算子使用的线性滤波器包含相邻的三行三列，以抵消简单（单行/单列）梯度滤波器的噪声敏感性
- Prewitt算子使用如下滤波器：

$$H_x^P = \begin{bmatrix} -1 & 0 & 1 \\ -1 & \mathbf{0} & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad \text{and} \quad H_y^P = \begin{bmatrix} -1 & -1 & -1 \\ 0 & \mathbf{0} & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

- Sobel算子与Prewitt算子的滤波器几乎相同，只是它为中间的行和列分别分配了更大的权值：

$$H_x^S = \begin{bmatrix} -1 & 0 & 1 \\ -2 & \mathbf{0} & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad \text{and} \quad H_y^S = \begin{bmatrix} -1 & -2 & -1 \\ 0 & \mathbf{0} & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

图像滤波器

- 这些滤波器对垂直或水平方向的边缘具有较强的响应，因此被认为是一种定向滤波器。
- OpenCV提供了cv::Sobel函数进行Sobel滤波：

```
cv::Sobel(image,          // 输入图像
          sobel,          // 输出图像
          image_depth,    //输出图像像素深度
          xorder, yorder, // x和y方向导数阶数
          kernel_size,  //正方形内核尺寸，必须为奇数
          alpha, // 缩放比例 (默认值为1)
          beta); // 偏移量(默认值为0)
```

图像滤波器

- 任务：Sobel滤波
- 读取并显示原图：

```
cv::Mat image = cv::imread("castle.jpg");
cv::imshow("原图", image);
```

- 转灰度图(一般针对灰度图进行Sobel滤波)

```
cv::Mat gray;
cv::cvtColor(image, gray, cv::COLOR_BGR2GRAY);
cv::imshow("灰度图", gray);
```

图像滤波器

- 水平方向Sobel滤波

```
cv::Mat sobelX;
cv::Sobel(gray,  // 输入图像
    sobelX,      // 输出图像
    CV_8U,       // 输出图像像素深度
    1, 0,        // x和y方向导数阶数
    3,           // 正方形内核尺寸，必须为奇数
    0.4, 128);   // 缩放比例和偏移量(默认值为1和0)
cv::imshow("水平方向Sobel滤波结果", sobelX);
```

Sobel滤波的结果有正有负，在将结果赋给输出图像中的像素时，会进行饱和处理，因此，为了能够将负值能够得到保留，使用了128偏移量以及缩放系数

图像滤波器

- 为了进行边缘检测，通常需要计算根据水平和垂直滤波的结果，计算Sobel滤波器的模：

```
// 计算Sobel滤波器的模
cv::Sobel(gray, sobelX, CV_16S, 1, 0);
cv::Sobel(gray, sobelY, CV_16S, 0, 1);
cv::Mat sobel;
// 计算L1模
sobel = abs(sobelX) + abs(sobelY);
```

为了保留滤波结果中所有原始值，输出图采用了16位有符号整数，且不使用缩放系数和偏移量

相对于L2模，L1模的计算速度更快

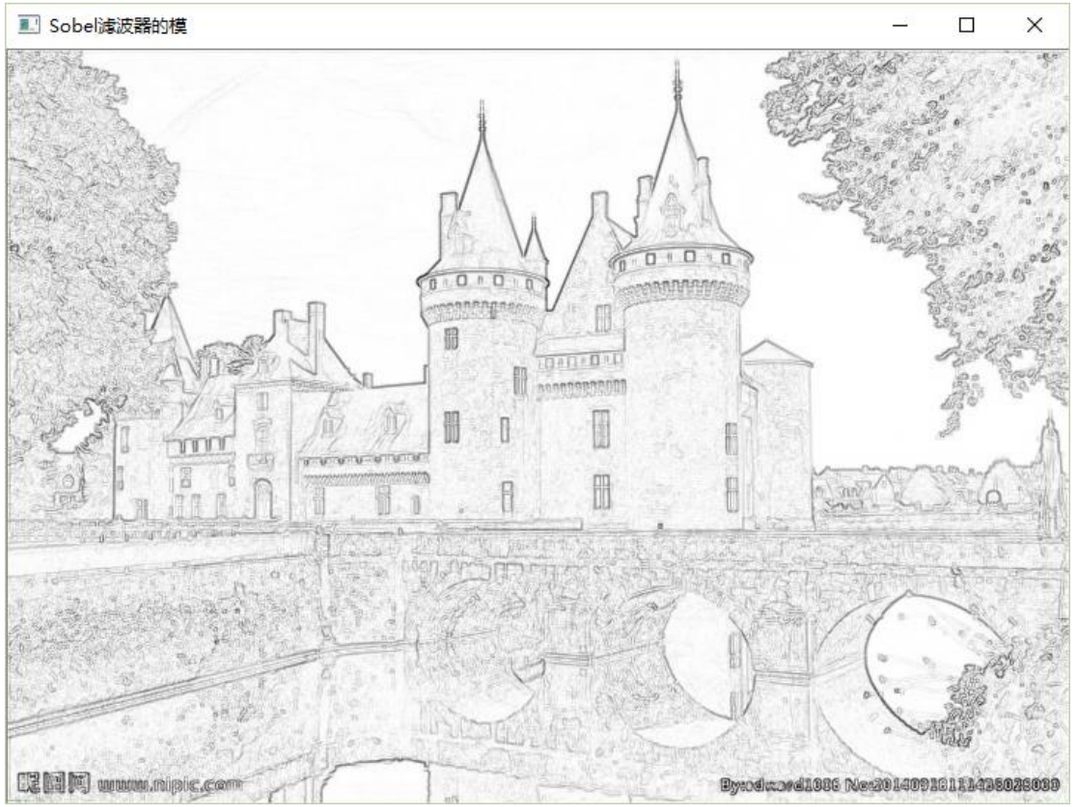
图像滤波器

- 为了更清晰地以图像形式显示Sobel滤波器的模，将其转为灰度图，并将最大模值对应为黑色：

```
// 找到Sobel中像素最大值
double sobmin, sobmax;
cv::minMaxLoc(sobel, &sobmin, &sobmax);
// 转换成8位图像
// sobelImage = -255. / sobmax * sobel + 255
cv::Mat sobelImage;
sobel.convertTo(sobelImage, CV_8U,
        -255. / sobmax, 255);
cv::imshow("Sobel滤波器的模", sobelImage);
```

图像滤波器

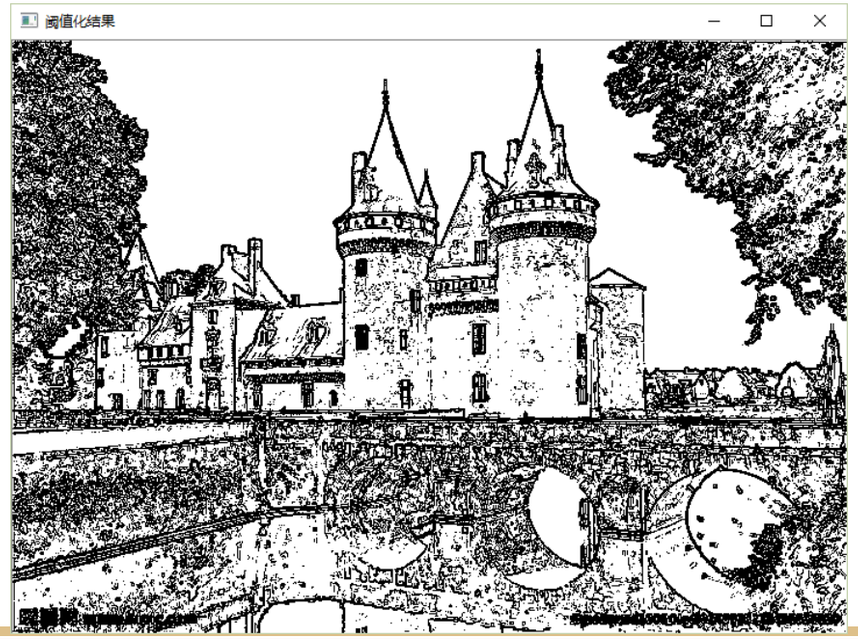
- Sobel滤波器的模



图像滤波器

- 接着可以对这个图像阈值化，得到图像轮廓的二值分布图

```
cv::Mat sobelThresholded;
double threshold = 230; // 阈值
cv::threshold(sobelImage, sobelThresholded,
      threshold, 255, cv::THRESH_BINARY);
cv::imshow("阈值化结果", sobelThresholded);
```



尝试使用不同的阈值看看

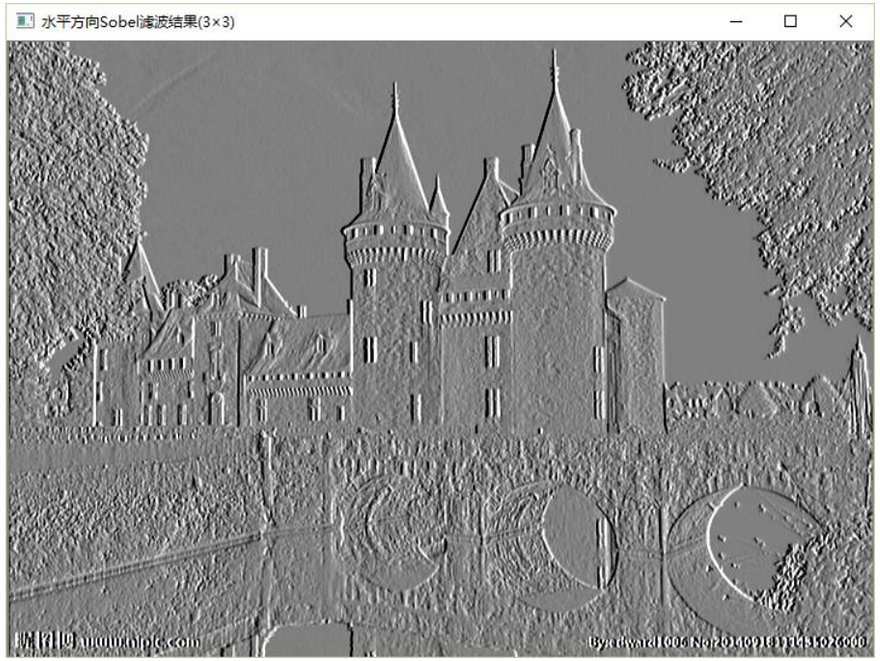
图像滤波器

- Sobel算子相当于先应用高斯平滑滤波，再应用Prewitt算子，如果采用大尺寸的Sobel滤波器，则平滑效果会更强
- 试试更大尺寸的Sobel滤波器看看：

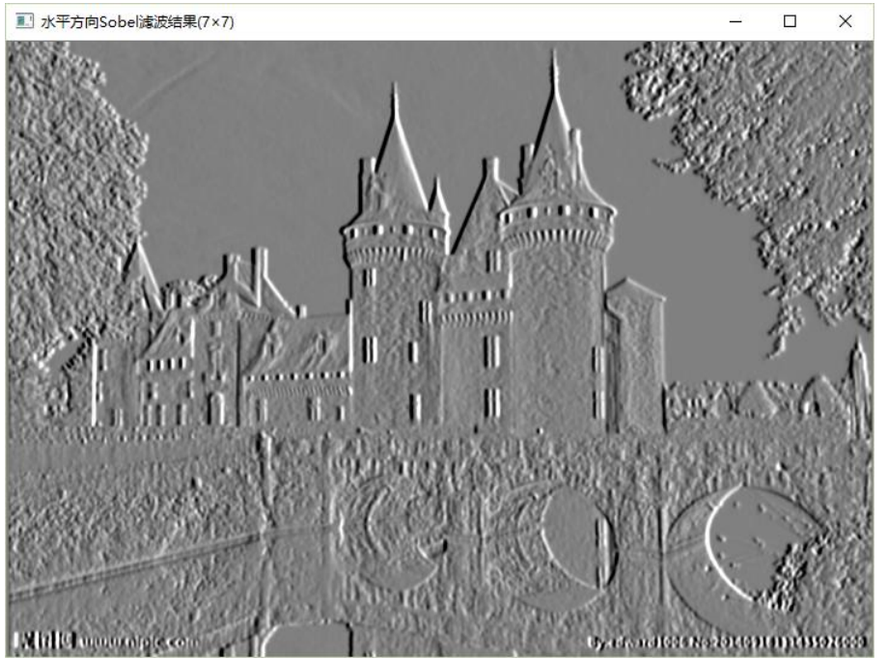
```
// 水平方向Sobel滤波
cv::Mat sobelX;
cv::Sobel(gray,        // 输入图像
          sobelX,      // 输出图像
          CV_8U,       // 输出图像像素深度
          1, 0,        // x和y方向导数阶数
          7,           // 正方形内核尺寸，必须为奇数
          0.002, 128); // 缩放比例和偏移量
```

图像滤波器

- 不同尺寸的Sobel滤波效果比较



3×3水平Sobel滤波



7×7水平Sobel滤波

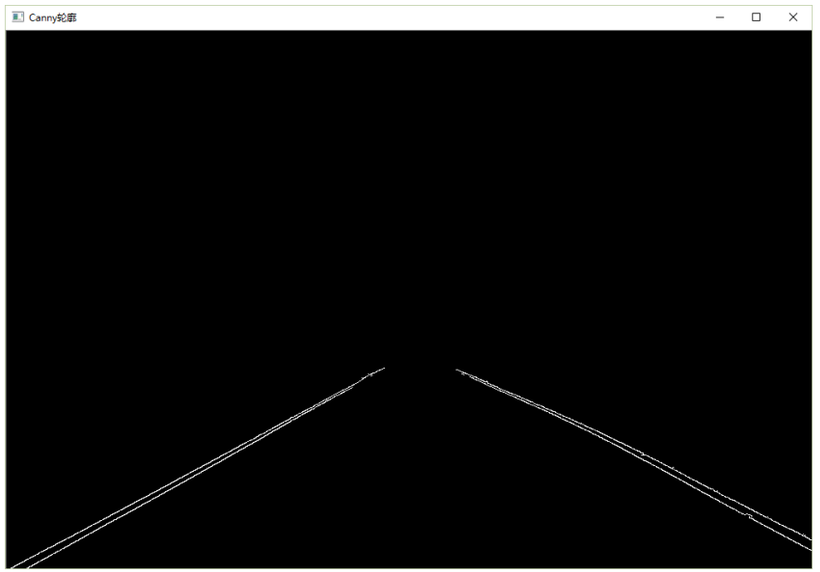
用Canny算子检测图像轮廓

- 利用之前的边缘检测方法所得到的二值边缘分布图有两个主要缺点：
  - 检测到的边缘过厚，这导致难以识别物体的边界
  - 通常不可能找到这样的阈值：低到足以检测到图像中所有重要的边缘，同时又高到足以避免产生太多无关紧要的边缘
- Canny算法可较好地解决此问题
  - 通常基于Sobel算子，也可使用其他梯度算子
  - 核心思想：用两个不同的阈值来判断哪个点属于轮廓
    - 一个低阈值
    - 一个高阈值

图像滤波器

- Canny算法可通过OpenCV的cv::Canny函数实现：

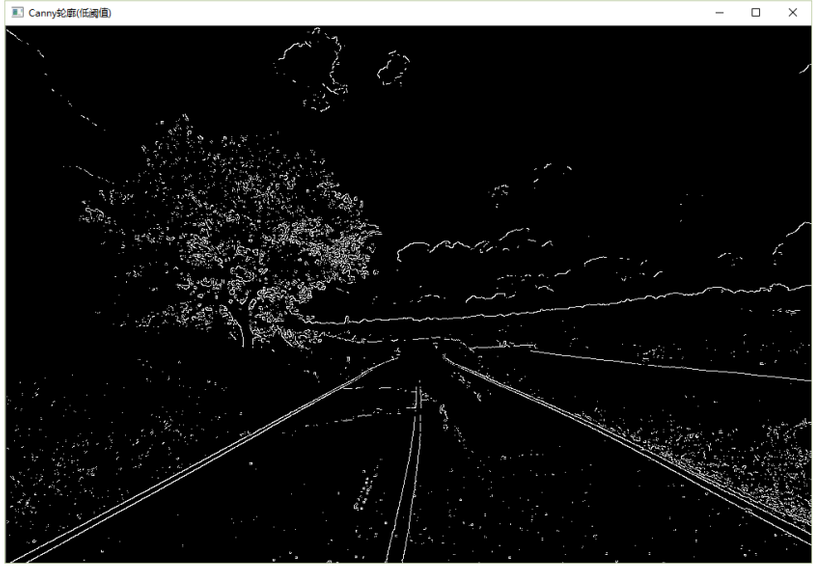
```
cv::Mat contours;
cv::Canny(gray,          // 灰度图像
          contours,      // 输出轮廓图
          350,           // 低阈值
          1250);         // 高阈值
```



图像滤波器

- 选择低阈值时，要保证它能包含属于重要图像轮廓的全部边缘像素
- 选择高阈值时，在不影响感兴趣边缘提取的前提下，尽可能地排除掉无关边缘
- 示例的目标：检测出道路边缘，尽可能排除无关边缘

仅用低阈值结果



仅用高阈值结果

图像滤波器

- Canny算法结合这两种边缘分布图，生成最优的轮廓分布图。具体做法是通过在强边缘（来自高阈值边缘分布图）周围寻找并连接弱边缘（来自低阈值边缘分布图）来保证轮廓的连通性，同时避免加入无关边缘
- 这种基于两个阈值获得二值分布图的策略，称为滞后阈值化
- 在进行滞后阈值化之前，如果梯度幅值不是梯度方向上的最大值，那么对应的边缘点会被移除，这就是为什么Canny轮廓分布图中边缘比较薄的原因

图像滤波器

- 任务：提取道路轮廓，尽量清除其他无关轮廓
- 步骤1：从百度图片搜索“道路”，选取一张作为测试图(不得使用PPT中的图)
- 步骤2：读入该图像，并利用cvtColor将图像转为灰度图，并将该图显示出来
- 步骤3：利用Canny函数，通过将高阈值参数设成和低阈值一样，测试低阈值的选取（在保证道路边缘连通性的前提下，尽可能地提高阈值）
- 步骤4：固定低阈值，测试不同的高阈值，在保证道路边缘连通性的前提下，尽可能地去除无关轮廓